

Algorithmische Methoden in der Praxis

Zusammenfassung

Henri Heyden

stu240825

Kiel University

Summer Term 2026

Inhaltsverzeichnis

1	Einleitung	4
1.1	Schnelles Parsen	4
2	Einfache und Abstrakte Datenstrukturen	4
2.1	Stack, Queue, und Deque	5
2.2	Liste	6
2.3	Bäume	6
2.3.1	Suchbäume	6
2.3.2	Heaps	7
2.4	Priority Queue	7
2.5	Graphen	7
2.6	Dictionaries	7
2.7	Sets	8
2.8	Union-Find	8
2.9	Datenstrukturen in Python	9
3	Sortieren und Suchen	9
3.1	Sortiervverfahren	9
3.2	Suchverfahren	10
3.3	Sortieren und Suchen in Python	11
4	Exhaustive Search und Backtracking	11
4.1	Permutationen Berechnen	11
4.2	Teilmengen Berechnen	12
4.3	Backtracking	12
5	Dynamische Programmierung	13
5.1	Top-Down	13
5.2	Bottom-Up	14
5.3	Dynamische Programmierung Anwenden	14
5.3.1	Top-Down versus Bottom-Up	14
5.4	Beispiel: SUBSET-SUM	14
6	Graphenalgorithmen	15
6.1	Suchen im Graphen	16
6.1.1	Tarjan's Algorithmus	17
6.1.2	Topologische Sortierung	17
6.2	Algorithmen auf Gewichteten Graphen	18
6.2.1	Kruskal's Algorithmus	18

6.2.2	Kürzeste Pfade	19
6.3	Flussnetzwerke	20
6.3.1	Maximum-Flow	20
6.3.2	Minimum-Cut	21
6.3.3	Maximum-Bipartite-Matching	22
7	Range Queries	22
7.1	Statische Daten	22
7.2	Dynamische Daten	23
7.2.1	Fenwick-Bäume	23
7.2.2	Segmentbäume	23

1 Einleitung

Die Vorlesung beschäftigt sich mit verschiedenen Programmiertechniken, welche angewendet werden können um algorithmisch zu optimieren. Man könnte also sagen, dass diese Vorlesung ähnlich wie *Hochleistungsrechnen* Programme beschleunigt, nur nicht durch die Hardware, sondern durch schlauere Algorithmen.

Die Techniken werden gerne in Programmierwettbewerben angewendet und gelehrt, weswegen sich die Vorlesung auch mit jenen auseinandersetzt.

In Programmierwettbewerben werden meistens Eingaben und Ausgaben, sowie Limits für die geheimen Eingaben bereitgestellt. Diese Einschränkungen können bereits ein guter Startpunkt sein, um einzuschätzen, wie langsam im worst-case ein naiver Algorithmus wäre.

1.1 Schnelles Parsen

Jeglicher Algorithmus muss erst mal die Eingabe lesen und parsen. Das erfolgt meistens über die Standard-Eingabe `stdin`. Meistens ist es schlau¹ das Laden der Eingabe komplett vor der Berechnung in einem Schritt zu vollziehen – in Python geht das zum Beispiel so:

```
1 from sys import stdin
2
3 tokens = iter(stdin.read().split())
```

Um nun einzelne Eingaben zu parsen, kann einfach auf den jeweils nächsten String mit `next(tokens)` zugegriffen werden, und evaluiert werden, bsp. `int(·)`.

2 Einfache und Abstrakte Datenstrukturen

Um komplexere Datenstrukturen zu verwenden kann es offenbar Vor- und Nachteile haben sie mithilfe anderer, einfacherer Datenstrukturen zu implementieren. Dabei sollte individuell abgewägt werden.

Statisches Array Ein klassisches Feld ermöglicht konstante Laufzeit beim Lesen und Schreiben eines Elementes, jedoch ist die Größe begrenzt und es muss manuell kopiert werden, wenn das Array vergrößert oder verkleinert werden soll.

Verknüpfte Elemente Eine *linked list* besteht aus einzelnen Elementen, die mithilfe von Referenzen auf irgendwie verbundene Elemente zeigen. Beispielsweise kann auf ein nächstes und vorheriges Element gezeigt werden, um einen Listenprototypen zu implementieren. Der Vorteil dieser Datenstruktur ist, dass in konstanter Laufzeit Elemente speichereffizient angehängt werden können, jedoch ist es nicht möglich konstant auf beliebige Elemente zuzugreifen. Dementsprechend lässt sich beispielsweise

¹Außer bei *ad-hoc* Problemen

binäre Suche nicht in $O(\log n)$ implementieren, sondern nur in $O(n)$. Außerdem entstehen offenbar kleine Verzögerungen bei der Speicherallokation und Deallokation, da dies nicht im Batch, sondern einzeln passiert.

Zweistufiges Array Alternativ zu einem einzelnen Array können auch die gleichen Daten in ein zweidimensionalen Array partitioniert werden, indem man das originale Feld in gleich große Teilfelder aufteilt. Das hat den Vorteil, dass man leichter Elemente in ein nicht ganz volles Teilfeld am Ende hinzufügen kann, man aber immer noch konstant schnell (in zwei Schritten) auf einzelne Elemente zugreifen kann.

Verknüpfte Arrays Hierbei handelt es sich um eine verkettete Liste, die selbst Felder speichert. Dies ist recht ähnlich zu dem zweistufigen Array, nur mit dem Unterschied, dass noch leichter hinzugefügt werden kann, jedoch der konstant schnelle Zugriff nicht mehr möglich ist.

Bucket List Eine Bucket List ist sozusagen das Gegenteil von einem verknüpften Array, wo stattdessen ein Feld einzelne verkettete Listen speichert.

2.1 Stack, Queue, und Deque

Mithilfe der definierten Datenstrukturen lassen sich abstrakte Datenstrukturen implementieren.

Stack In einem Stack werden Daten auf bzw. vom Speicher gelegt. Hierfür werden die Prototypen `top()`, `pop()`, und `push(x)` implementiert, die jeweils das oberste Element zurückgeben, das oberste Element entfernen und zurückgeben, und ein Element auf den Stack legen.

Queue In einer Queue werden Daten hinten an den Speicher gelegt, und vorne entfernt. Es werden die Prototypen `front()`, `pop()`, und `push(x)` implementiert, die jeweils das vorderste Element zurückgeben, das vorderste Element entfernen und zurückgeben, und ein Element hinten an die Queue anfügen.

Deque Eine Deque ist eine Datenstruktur, die gleichzeitig Queue und Stack realisiert.

Alle Operationen dieser Datenstrukturen sind in konstanter Laufzeit implementierbar mithilfe verketteter Listen, wenn man Zeiger zum ersten/letzten Element speichert, die automatisch aktualisiert werden. Alternativ ist es aber auch möglich, wenn man eine maximale Größe spezifiziert, die Datenstrukturen mithilfe statischer Arrays zu implementieren.

2.2 Liste

Eine weitere abstrakte Datenstruktur ist die Liste, welche das Lesen, Einfügen, und Löschen an beliebigen Positionen realisieren soll.

Hierbei gibt es mehrere Möglichkeiten wieder diesen Typen zu implementieren. Verwendet man eine verkettete Liste ist das Einfügen/Löschen in $O(1)$ und der Zugriff in $O(n)$ implementierbar. Verwendet man stattdessen naiv ein statisches Array, ist das Einfügen/Löschen in $O(n)$ und der Zugriff in $O(1)$ realisierbar.

Es ist aber möglich eine amortisierte Laufzeit von $O(1)$ für das Einfügen/Löschen am Ende zu erreichen mit einem statischen Array, indem das Feld regelmäßig um einen Faktor vergrößert bzw. verkleinert wird, sodass Elementen selten kopiert werden müssen – das nennt man dann auch ein *dynamisches Array*. Übliche Faktoren zum Vergrößern und Verkleinern liegen zwischen 1.5 und 2.

2.3 Bäume

Allgemein sind Bäume zusammenhängende Graphen ohne Zyklen. Sie können über verknüpfte Elemente dargestellt werden. Dabei können einzelne Elemente unterschiedlich benachbarte Elemente indizieren. Beispielweise kann jedes Element seinen Elternknoten speichern, oder jedes Element speichert seine Kinder. Diese Implementierungen nennt man *bottom-up* und *top-down*.

Es ist wieder auch möglich (binäre) Bäume mithilfe von statischen Arrays zu implementieren, indem die einzelne Ebenen des Baumes als Array aufgezählt werden. Somit kann für einen Knoten i die Position eines Vorgängers durch $\lfloor i/2 \rfloor$, sowie die Kinder mit $2i$ und $2i + 1$ bestimmt werden.

2.3.1 Suchbäume

Enthalten einzelne Knoten einen Schlüsselwert, lassen sich (binäre) Suchbäume definieren dadurch, dass für alle Elemente der Wert des linken Kindknotens kleiner ist als der Schlüsselwert, der nun wieder kleiner ist als der Wert des rechten Knotens.

Hier existieren spezielle balancierende Varianten, wie den Rot-Schwarz-Baum, wo das Suchen, Einfügen, und Entfernen im Baum in $O(\log n)$ implementiert werden kann.

2.3.2 Heaps

Heaps sind spezielle Bäume, bei denen für alle Knoten einer Ebene gilt, dass sie kleiner (*Min-Heap*) bzw. größer (*Max-Heap*) sind als ihre Kinder.

In Heaps wird das Einfügen eines Elementes sowie das Entfernen der Wurzel implementiert. Das ist möglich in jeweils $O(\log n)$, durch eine Helfer-Funktion *heapify*, bei der jeweils Elemente zwischen Ebenen getauscht werden, bis die Heap-Eigenschaft wiederhergestellt ist. Beim Entfernen wird dabei die Wurzel entfernt und das letzte Element des Heaps in die Wurzel eingefügt, nachdem *heapify* auf die Wurzel losgelassen wird. Beim Einfügen passiert das gleiche, nur, dass das neue Element versucht wird an der höchsten Ebene eingefügt zu werden. Muss ein Element getauscht werden, wird mit dem verschobenen Element auf die gleiche Art weitergemacht.

2.4 Priority Queue

Die Priority Queue ist eine Queue, bei der ein Schlüsselwert (Priorität) bei einzelnen Elementen bestimmt, welche Elemente in welcher Reihenfolge aus der Queue entfernt werden. Hier werden (bei einer Max-Priority Queue) die Prototypen `maximum()`, `extractMax()`, und `insert(x, p)` realisiert.

Offenbar lassen sich Priority Queues einfach schnell mithilfe von Heaps implementieren. Dann kommt man auf $O(\log n)$ für `insert` und `extractMax`, sowie $O(1)$ für `maximum`.

2.5 Graphen

Graphen beschreiben Relationen zwischen einzelnen Objekten. Meistens werden sie definiert als ein Tupel (V, E) mit $V = [n]$ für $n \in \mathbb{N}$ und $E \subseteq V^2$ oder $E \subseteq \{\{v, w\} \subset V \mid v \neq w\}$ je nachdem, ob man gerichtete oder ungerichtete Graphen behandelt². Graphen können auch gewichtet sein, das bedeutet, dass man eine Gewichtung $w \in \mathbb{R}^E$ definiert. Außerdem gibt es zyklische und azyklische Graphen, je nachdem wenn es einen Pfad im Graphen gibt, der den Ursprungsknoten erreicht. Siehe section 6 für Algorithmen auf Graphen.

2.6 Dictionaries

Ein Dictionary stellt eine Abbildung zwischen Schlüsseln und Werten dar. Dabei sind die Typen von Schlüsseln und Werten nicht beschränkt! Es werden die Prototypen `find(k)`, `erase(k)`, und `insert(k, v)` realisiert, die jeweils einen Wert durch seinen Schlüssel finden, ein Wert/Schlüssel-Paar mithilfe des Schlüssels löschen, und ein Wert/Schlüssel-Paar einfügen.

Dictionaries lassen sich durch Suchbäume implementieren, wobei alle Operationen in $O(\log n)$ möglich werden, wobei angenommen werden muss, dass der Schlüssel-Typ vergleichbar ist.

²Es gibt auch Graphen mit Mehrfachkanten oder Selbstkanten.

Alternativ lassen sich Dictionaries über sog. *Hash-Tabellen* implementieren, wo ein Schlüsselwert in einen numerischen Typ umgewandelt wird, welcher dann als Index in einer Bucket List den Bucket mit dem gesuchten Wert/Schlüssel Paar findet. Dann muss in diesem Bucket nach dem Paar gesucht werden. Meistens sind jegliche Operationen in $O(1)$ nutzbar, jedoch ist es möglich, dass es zu Kollisionen kommt, bei einer Hash-Funktion, wodurch es zu $O(n)$ bei nicht-vergleichbaren, und $O(\log n)$ bei vergleichbaren Daten³ kommen kann. Um die Kollisionen zu reduzieren, lässt sich auch die Menge an Buckets vergrößern, was dann natürlich leider den benötigten Speicher stark steigen lassen kann.

2.7 Sets

Mengen oder eng. *Sets* stellen die folgenden Prototypen für Werte eines Typen dar: `find(A, x)` gibt das Element zurück, wenn es in der Menge ist. `erase(A, x)` entfernt das Element von der Menge. `insert(A, x)` fügt das Element in der Menge hinzu. `union(A, B)` vereinigt beide Mengen. `cut(A, B)` berechnet die Schnittmenge. `diff(A, B)` berechnet die Differenz der Menge zur Zweiten.

Mengen lassen sich als Dictionary implementieren, wobei der Wert als Schlüssel verwendet wird.

2.8 Union-Find

Die Union-Find, auch *disjoint-set*, ist eine Datenstruktur, die eine Partition einer Menge speichert. Sie bietet folgende Prototypen an: `makeSet(x)` erzeugt eine Menge mit einem Element. `findSet(x)` gibt die Menge zurück, in welcher das Element sich befindet. `union(x, y)` vereinigt die Mengen der beiden Elemente.

Union-Find kann mithilfe von verketteten Listen implementiert werden, dann käme man auf $O(1)$ für `makeSet(x)`, $O(1)$ für `findSet(x)`, und $O(n)$ für `union(x, y)`. Das funktioniert so, indem man in jedem Element immer einen Zeiger zum Repräsentanten seiner Menge gibt, und dann beim Vereinigen jedes Element modifiziert, sodass sie auf den gleichen Kopf zeigen. Das kann optimiert werden, wenn man zusätzlich die Größe der Mengen speichert, und jeweils die kleinere Menge an die größere Menge anhängt.

Alternativ lässt sich Union-Find mithilfe von balancierten Bäumen implementieren. Da ist die Idee einfach normal in dem Suchbaum zu suchen für `findSet(x)` und eine spezielle `join`-Operation zu verwenden, um zwei Bäume miteinander zu verbinden und automatisch zu balancieren. Dann kann man $O(\log n)$ für beide Operationen erreichen.

Die *beste* Implementierung von Union-Find verwendet Pfadkompression, dann kommt man auf $O(n + m \cdot \alpha(n))$ für m `findSet(x)` und n `union(x, y)` Aufrufe, wobei α die Umkehrfunktion der Ackermann-Funktion ist.

³Bei Kollisionen kann man einfach in einem Suchbaum suchen bzw. jenen erweitern.

2.9 Datenstrukturen in Python

In Python sind einige Datenstrukturen schon vordefiniert.

```
1 # Stack und Queue/Deque
2 list | collections.deque
3 # Priority Queue als Liste mit Heap-Methoden
4 heapq.heappush(list, item)
5 heapq.heappop(list)
6 # Liste, Achtung: in mit Laufzeit  $O(n)$ 
7 list
8 # Menge, in mit durchschnittlicher Laufzeit  $O(1)$ , union und difference aber mit  $O(n)$ 
9 set
10 # Dictionary
11 dict
```

3 Sortieren und Suchen

Oft eignet es sich gut, besonders bei Greedy-Algorithmen, eine Eingabe nach einer Ordnungsrelation zu sortieren. Beispielweise möchte man ein statisches Array nach Prioritäten sortieren um eine Reihenfolge der Bearbeitung festzulegen. Mediane und Quartile eines Datensatzes lassen sich weitaus leichter berechnen, wenn zunächst sortiert wird. Oder um festzustellen, ob Duplikate in einem Feld sind, lässt sich das Feld sortieren, um dann ein mal über sie iterieren. So ähnlich lassen sich auch Häufigkeiten von Elementen herausfinden, oder Duplikate löschen⁴. Eine ursprüngliche Reihenfolge kann erreicht werden, indem man vor dem Sortieren die Position als Attribut jedem Element hinzufügt, und nach der Bearbeitung wieder nach diesem Attribut sortiert. Außerdem können sortierte Felder verwendet werden, um für unterschiedliche Werte und einen Zielwert Paare zu finden, die diesen Zielwert aufsummiert oder multipliziert erreichen. Dabei wird ein Mal gleichzeitig von links und von rechts durch das Feld iteriert, um alle validen Paare zu finden⁵.

Ein sortiertes Feld kann aber sehr effektiv für die Suche verwendet werden, denn sie ermöglicht Binäre Suche, womit in $O(\log n)$ ein Wert gefunden werden kann.

3.1 Sortierverfahren

Es existieren offenbar verschiedene Algorithmen, um ein Feld $X \in \mathbb{R}^{[n]}$ zu sortieren.

Selectionsort Suche das Minimum aus dem Feld und füge es links ein. Erreicht $O(n^2)$, ist allgemein nicht stabil, jedoch in-place.

⁴Verwende zwei Pointer die jeweils durch das Feld durchlaufen und auf das letzte nicht gelöschte Element zeigen. Dann werden Duplikate mit dem jeweils nächsten Element vertauscht.

⁵Wenn das Paar zu groß ist, wird rechts iteriert, sonst links.

Insertionsort Teile Feld in sortierten und nicht-sortierten Teil auf. Füge immer wieder ein Element aus dem nicht-sortierten Teil in den sortierten Teil ein. Erreicht $O(n^2)$, ist stabil und in-place.

Bubblesort Gehe n mal durch das Feld durch und Tausche Elemente, wenn sie an der falschen Position sind. Sortiere weiter mit dem getauschten Element. Erreicht $O(n^2)$, ist stabil und in-place.

Quicksort Wähle ein Pivot-Element aus dem Feld, schiebe alle kleineren Elemente rechts von dem Element nach Links und andersrum. Sortiere links und rechts rekursiv weiter. Erreicht $O(n \log n)$, ist nicht stabil, jedoch in-place.

Mergesort Teile linke und rechte Hälfte rekursiv auf, füge sortierte Hälften ineinander. Erreicht $O(n \log n)$, ist stabil, jedoch meistens nicht in-place.

Heapsort Füge alle Elemente in einen Heap und poppe n mal den Baum. Erreicht $O(n \log n)$, ist nicht stabil, jedoch in-place.

Man kann auch Felder sortieren ohne Vergleiche zu verwenden, dann sinkt die obere Schranke von $\Omega(n \log n)$ auf $\Omega(n)$.

Bucketsort Verteile Elemente durch monotone Abbildung der Werte/Schlüssel auf k Buckets und sortiere einzelne Buckets weiter. Im Fall $k = n$ erreicht man $O(n)$. Stabil möglich, aber nicht in-place.

Countingsort Für Feld X sei $k := \max(X)$. Für Feld der Größe k zähle wie oft jedes $i \in [k]$ vorkommt, und schreibe diese Zahl in das Feld. Berechne nun aus diesem Feld das sortierte Feld. Erreicht $O(n + k)$ sinnvoll zu nutzen für $k \ll n$. Ist stabil, jedoch nicht in-place.

3.2 Suchverfahren

Es gibt auch unterschiedliche Algorithmen um in einem Feld $X \in \mathbb{R}^{[n]}$ nach $x \in \mathbb{R}$ zu suchen.

Sequenzielle Suche Gehe linear durch X durch und stoppe wenn $x = X[i]$. Laufzeit $O(n)$. Wenn X sortiert ist, kann man vorzeitig stoppen, wenn $x < X[i]$.

Binäre Suche Wenn X sortiert ist, kann man immer wahlweise die Mitte $m = n/2$ wählen, und dann, wenn das Element kleiner ist, weitersuchen in $X[..m - 1]$, bzw. wenn es größer ist in $X[m + 1..]$, bis es gefunden wurde. Laufzeit $O(\log n)$.

Der Algorithmus lässt sich auch auf reelle Intervalle übertragen, beispielweise bei der Suche nach Nullstellen einer Funktion – das nennt man *Bisektion*.

3.3 Sortieren und Suchen in Python

Python liefert verschiedene Standardfunktionen um zu suchen und zu sortieren.

```
1 | # Sortiert liste stabil,  $O(n \log n)$ 
2 | list.sort()
3 | # Erstellt einen Heap,  $O(n)$ 
4 | heapq.heapify(list)
5 | # Hinzufügen und Entfernen aus einem Heap,  $O(\log n)$ 
6 | heapq.heappush(list, item)
7 | heapq.heappop(list)
8 | # Binäre Suche,  $O(\log n)$ 
9 | bisect.bisect(list, item)
```

4 Exhaustive Search und Backtracking

Eine klassische Lösungsmethode jedes Problems ist die exhaustive Suche, bei der kontinuierlich alle Lösungskandidaten berechnet und getestet werden. Dies benötigt das Aufzählen aller Permutationen bzw. Teilmengen des Inputs. Bei einigen NP-vollständigen Problemen muss wahrscheinlich leider auf diese Methode zurückgegriffen werden, was meistens in $O(2^n)$ resultiert.

Etwas klüger funktioniert das Backtracking, indem der Suchraum rekursiv beschränkt wird und dabei so eingeschränkt wird, dass vorzeitig abgebrochen werden kann, wenn eine Lösung nicht mehr optimal werden kann.

4.1 Permutationen Berechnen

Eine Möglichkeit um alle $n!$ Permutationen einer Eingabe $X \in \mathbb{R}^n$ zu berechnen ist *Heap's Algorithmus*:

```
1 | def HeapsAlgorithm(X, n):
2 |     if n == 1:
3 |         test(X) # Probiere aktuelle Permutation
4 |     else:
5 |         for i in range(n-1):
6 |             HeapsAlgorithm(X, n-1)
7 |             if n % 2 == 0:
8 |                 X[i], X[n-1] = X[n-1], X[i]
9 |             else:
10 |                 X[0], X[n-1] = X[n-1], X[0]
11 |         HeapsAlgorithm(X, n-1)
```

Wenn die Reihenfolge, in der die jeweiligen Permutationen getestet werden sollen wichtig ist, muss offenbar deutlich mehr getauscht werden. In Python kann effizient einfach `itertools.permutations(X, len(X))` verwendet werden um alle Permutationen von X der Länge n in lexikographischer Reihenfolge zu finden.

4.2 Teilmengen Berechnen

Die Teilmengen von X lassen sich leicht berechnen, indem man einfach in einem Bitvektor der Länge n durch alle möglichen Werte geht:

```
1 def Subsets(X: set):
2     for b in range(2 ** n):
3         A = set()
4         for i in range(n):
5             if b & (2 ** i) != 0:
6                 A += set(X[i])
7     test(A)
```

4.3 Backtracking

Backtracking heißt eine schrittweise Konstruktion der richtigen Lösung durch Auswahl eines Elements. Wenn ein Misserfolg erkannt wird, wird der letzte Zug rückgängig gemacht, um dann verwandte Alternativen auszuprobieren.

Beispielweise kann Backtracking im SUBSET-SUM Problem verwendet werden, bei den für $X \subseteq \mathbb{N}$, $n = |X|$ und $M \in \mathbb{N}$ eine Teilmenge $S \subseteq X$ mit $\sum S = M$ gesucht wird. Dabei wird der Suchbaum durchgegangen, und vorzeitig abgebrochen, wenn die derzeit gesammelten Elemente schon bereits M überschreiten. Dann wird zum letzten Knoten zurückgegangen und weiter gesucht. Somit kann der Suchbaum deutlich verkleinert werden – das nennt man auch *search tree pruning*. Meistens reduziert diese Strategie die Laufzeit, jedoch wurde in diesem Beispiel der worst-case nicht verbessert, wenn keine Teilmenge die Summe erreicht.

Das Lösungsschema lässt sich auf einen grundlegenden Ablauf runterbrechen:

1. Starte mit leerer Lösung, $s = []$; `Backtracking(s)`. Wiederhole folgendes bis Abbruch:
2. Teste ob der derzeitige Knoten eine Lösung ist, `IsSolution(s)`.
3. Teste ob es noch Lösungskandidaten gibt weiter im Suchbaum von hier aus, `IsValid(s + [c])` für mögliche Schritte c .
 - Wenn ja, gehe weiter im Suchbaum von dieser Stelle aus, `s.push(c)`; `Backtracking(s)`.
 - Wenn nicht, kehre zum letzten Kandidaten zurück, von dem es aus noch andere Lösungskandidaten gab, `s.pop()`

Anwendungsbeispiele für Backtracking sind u.a., kombinatorische Probleme, KNAPSACK, SUBSET-SUM, das Damenproblem, Sudoku, Pfadsuche, oder Graphenfärbung.

Backtracking lässt sich meistens auch noch besser optimieren, indem man jeweils Kandidaten wählt, von denen man nach einer Greedy-Strategie erwartet, dass sie besser sein könnten als andere – eine Form von *look-ahead*.

Sind jedoch minimal lange Lösungen gesucht, sollte man vielleicht Breitensuche statt der klassischen Tiefensuche verwenden.

5 Dynamische Programmierung

Dynamische Programmierung (DP) ist ein Ansatz, indem eine Rekursion optimiert wird. Das Hauptziel ist es, die Lösung aus den Lösungen für kleinere Probleme aufzubauen, welche genau ein mal berechnet werden. Offenbar ist nicht jeder rekursiver Algorithmus eine Anwendung von dynamischer Programmierung, berechnet man naiv rekursiv die Fibonacci-Sequenz, berechnet man viele Vorgänger mehrfach.

Um dynamische Programmierung anwenden zu können, muss das Problem natürlich mit *Teile-und-Herrsche* lösbar sein, da offensichtlich die Lösung überhaupt strukturell aufbaubar aus Teillösungen sein muss. Dazu lohnt es sich natürlich auch nur, dynamische Programmierung anzuwenden, wenn beim Lösen des Problems mehrere Teillösungen mehrmals berechnet werden müssen. Wenn dies nicht der Fall ist, kann man einfach naiv rekursiv programmieren, ohne bestraft zu werden.

Dynamische Programmierung lässt sich grundlegend in zwei verschiedene Ansätze unterbrechen: *Top-Down* und *Bottom-Up*.

5.1 Top-Down

Top-Down basiert auf Memoisation, bei der rekursiv berechnet wird, und wichtige Zwischenergebnisse zwischengespeichert werden, wenn sie noch nicht berechnet wurden. Der Name *Top-Down* entspricht daher, dass durch die Rekursion der Berechnungsgraph vom initialen Aufruf vollständig aufgefallen wird.

Beispielsweise kann so die n -te Fibonacci Zahl mit Top-Down berechnet werden:

```
1 F = []
2 def fib(n):
3     global F
4     if not F:
5         F = [0, 1] + [None for i in range(n - 1)]
6     if not F[n]:
7         F[n] = fib(n-1) + fib(n-2)
8     else:
9         return F[n]
```

Hier wird ein statisches Feld `F` verwendet, welches die trivialen Ergebnisse speichert, und in welches jegliche Zwischenergebnisse eingefügt werden, wenn sie nicht berechnet wurden.

5.2 Bottom-Up

Im Vergleich zu Top-Down wird bei Bottom-Up stattdessen Tabulation verwendet, bei der alle Zwischenergebnisse iterativ direkt berechnet werden, bevor die eigentliche Instanz berechnet wird. Ein typisches Beispiel dazu ist die Levenshtein-Distanz, oder wieder die Fibonacci Sequenz:

```
1 def fib(n):
2     F = [0, 1] + [None for i in range(n - 1)]
3     for i in range(1, n+1):
4         F[i] = F[i-1] + F[i-2]
```

Falls oft diese Funktion aufgerufen werden soll, kann es sich empfehlen sie vorzuberechnen und einfach $F[n]$ direkt zurückzugeben.

5.3 Dynamische Programmierung Anwenden

Um dynamische Programmierung in einem Algorithmus anzuwenden muss man zunächst, wie beim Backtracking, die Zustände identifizieren, welche Probleminstanzen beschreiben – beim Fibonacci Problem sind das alle $F(i)$ für Entfernung $i \in [n]$ von der Wurzel. Danach müssen die Zustandsübergänge gefunden werden, die die jeweiligen Teillösungen erweitern – der Induktionsschritt, bei Fibonacci $F(i) = F(i - 1) + F(i - 2)$. Offenbar müssen dafür die Randfälle definiert werden – die Induktionsanfänge, bei Fibonacci $F(0) = 0, F(1) = 1$. Zuletzt muss entschieden werden, ob Top-Down oder Bottom-Up verwendet werden soll.

5.3.1 Top-Down versus Bottom-Up

Es gibt Vor- und Nachteile beider Ansätze. Top-Down ist leichter intuitiv umzusetzen, da sie den naiven rekursiven Algorithmus erweitert, jedoch entsteht durch die verwendeten Rekursionen ein Overhead. Im Vergleich nutzt Bottom-Up durch die iterative Berechnung gut Caching aus, jedoch werden alle Zwischenergebnisse berechnet, und nicht nur die nötig zu berechnenden.

5.4 Beispiel: SUBSET-SUM

Beim SUBSET-SUM Problem lässt sich dynamische Programmierung anwenden, um das Problem zu optimieren. Sei eine Probleminstanz $x \in \mathbb{N}^n, n \in \mathbb{N}$ mit Zielwert $m \in \mathbb{N}$. Gesucht ist die kleinste⁶ Teilmenge $S \subseteq \{x_i\}_{i \in [n]}$ mit $\sum S = m$.

Wir definieren Zustände $T(i, j) :=$ Größe der kleinsten Teilmenge von $\{x_k\}_{k \in [i]}$ für $i \in [n]$ mit Summe $j \in [m]$. Somit definieren sich die Randfälle $T(i, 0) = 0$ und $T(0, j) = \infty$ für $j > 1$.

⁶Hier als Optimierungsproblem.

Die Rekursion definiert sich dadurch, dass wir entweder ein neues Element x_i beim Iterieren über i verwerfen müssen, oder sonst entscheiden, ob es sich lohnt zu nehmen.

$$T(i, j) := \begin{cases} T(i-1, j) & \text{wenn } x_i > j, \\ \min(T(i-1, j), T(i-1, j-x_i) + 1) & \text{sonst}^7. \end{cases}$$

Je nachdem ob Memoisation oder Tabulation verwendet wird, wird beim Lösen $T(n, m)$ berechnet. Um dann die minimale Teilmenge zu finden, die mit dem Ergebnis assoziiert wird, muss man nur die Schritte zu Lösung einzeln noch mal machen und dabei sich die gewählten Elemente merken.

Dynamische Programmierung lässt sich wie gesagt gut anwenden für die Fibonacci Sequenz, die Levenshtein Distanz, SUBSET-SUM, oder KNAPSACK, der Longest Common Subsequence, Longest Increasing Subsequence, dem Longest Common Substring, oder dem TRAVELLING-SALESMAN Problem.

6 Graphenalgorithmen

Für ungerichtete Graphen V, E definieren wir die Adjazenzmenge $\text{adj} : V \rightarrow \mathcal{P}(V), v \mapsto \{u \in V \mid \{u, v\} \in E\}$ und den Grad eines Knoten $\text{deg}(v) : V \rightarrow \mathbb{N}, v \mapsto |\text{adj}(v)|$.

Für gerichtete Graphen definieren wir analog $\text{next}(v)$ und $\text{pred}(v)$ als die Menge von eingehenden und ausgehenden Knoten, mit Kardinalitäten $\text{deg}_{in}(v)$ und $\text{deg}_{out}(v)$.

Graphen lassen sich auf unterschiedliche Weisen modellieren.

Adjazenzmatrix Stelle Verbindungen von Knoten dar als Einträge in Matrix $A \in \{0, 1\}^{n \times n}$ oder $A \in \mathbb{R}^{n \times n}$ für gewichtete Graphen. $A_{ij} \neq 0$ genau dann, wenn $\{i, j\} \in E$, analog für gerichtete Graphen. Die Adjazenzmatrix hat den Vorteil, dass in konstanter Laufzeit auf eine Kante zugegriffen werden kann. Der Nachteil ist der exzessive Speicherverbrauch in $\Theta(n^2)$ und, dass $O(n)$ gebraucht wird, um alle Nachbarn eines Knoten herauszufinden.

Adjazenzliste Alternativ kann man alle benachbarten Knoten in einer Form von einer Bucket List speichern, bei der ein Feld die jeweiligen Adjazenzmengen der Knoten indiziert. In Python eignet sich sehr gut `list[set]` dafür, da man bei einem Speicherverbrauch $\Theta(|V| + |E|)$ sogar amortisiert konstant schnell auf eine Kante $\{i, j\}$ zugreifen kann⁸. Wenn man aber lieber schnell auf Nachbarn iterieren will, und so ein Lookup nicht so wichtig ist, sollte man doch lieber `list[list]` verwenden, da `set` langsamer als Listen iterieren.

⁷Linker Term: kleinste Teilmenge ohne x_i . Rechter Term: kleinste Teilmenge, die x_i wählt.

⁸Verwende man naiv `list[list]`, müsste man wirklich durch $\text{deg}(v)$ Knoten maximal durchgehen, da `in` bei Listen $O(n)$ braucht, jedoch nur $O(1)$ durchschnittlich bei `set`.

Verknüpfte Datenstruktur Ähnlich wie bei typischen Implementierungen für Baumstrukturen, lassen sich Graphen auch als Datentypen speichern, welche Knoten als Zeigern auf andere Knoten definieren. Das hat den Vorteil, dass man direkt die Topologie des Graphens abbildet, jedoch kann die Navigation im Graphen umständlich sein.

Manchmal definiert sich die Topologie des Graphens geometrisch. Beispielweise wenn Knoten Punkte im Raum sind, und eine Kante zwischen Knoten genau dann liegt, wenn sie nah genug sind. Oder wenn in einem Gitter Zellen dargestellt werden, die benachbart sind, wenn ihre Manhattan-Distanz genau 1 ist. In diesen Fällen sollte man eine Datenstruktur wählen, die diese Topologie gut abbildet, wenn sie relevant zum Problem ist.

6.1 Suchen im Graphen

Es gibt verschiedene Algorithmen, um in einem Graphen zu suchen oder Knoten zu besuchen und zu verarbeiten. Die bekanntesten Algorithmen hierfür sind die Tiefensuche und die Breitensuche.

Tiefensuche Wähle irgendeine Kante des derzeitigen Knotens und besuche den Nachbarn. Wenn der Knoten keine unbesuchten Nachbarn besitzt, gehe zurück zum letztes besuchten Knoten mit neuen Nachbarn. Besuche alle Nachbarn in FIFO-Reihenfolge (Stack).

Breitensuche Besuche direkt nacheinander alle unbesuchten Nachbarn des aktuellen Knotens. Mache dann beim zuerst besuchtem Nachbarn weiter. Besuche alle Nachbarn in LIFO-Reihenfolge (Queue).

Graphensuche ist oft eine Grundlage, um kürzeste Wege zu ermitteln⁹, Zusammenhangskomponenten zu ermitteln, topologisch zu sortieren, oder Zyklen zu finden.

```
1 adj: list[list]
2 b = [False for v in V]
3
4 def dfs(v):
5     """Rekursive Tiefensuche"""
6     global b
7     b[v] = True
8
9     # Mache irgendwas mit dem aktuellen Knoten
10    processNode(v)
11
12    for w in adj[v]:
13        if not b[w]:
14            dfs(w)
15
```

⁹Breitensuche findet sogar den kürzesten Pfad in ungewichteten Graphen.

```

16 def dfs_or_bfs_iter(root):
17     """Iterative Tiefen- bzw. Breitensuche"""
18     Q = collections.deque()
19     Q.push(root)
20     b[root] = True
21     while Q:
22         # Q.popleft() bei Breitensuche
23         v = Q.pop()
24
25         processNode(v) # ...
26
27         for u in adj(v):
28             if not b[u]:
29                 Q.push(u)
30                 b[u] = True

```

6.1.1 Tarjan's Algorithmus

Man kann die Schnittknoten¹⁰ bestimmen indem man Tiefensuche clever erweitert durch zwei Felder *disc* und *low*. *disc* speichert für jeden Knoten den Zeitpunkt wann der Knoten erstmals besucht wurde. *low* speichert die Anzahl an Vorfahrer-Knoten, die vom Knoten aus erreichbar sind, ohne seinen Elternknoten zu besuchen, speichern.

Alle Knoten, die nun im generiertem Baum der Tiefensuche entweder eine Wurzel mit mehr als einem Kind, oder ein Kind mit *low* größer/gleich *disc* seines Elternknotens, sind Schnittknoten. Dieser Algorithmus heißt *Tarjan's Algorithmus*.

6.1.2 Topologische Sortierung

Für gerichtete azyklische Graphen (DAGs) gibt es offenbar eine Reihenfolge $p \in [n]^V$, sodass $(v, w) \in E \Rightarrow p_v < p_w$. Die Reihenfolge der Knoten nennt man topologische Sortierung – eine eines Graphen zu finden geht mit Kahn's Algorithmus. Manchmal kann das Traversieren/Suchen des Graphens in dieser Reihenfolge praktisch sein, da eine abgeleitete Hierarchie eingehalten wird.

```

1 nexT = list[list]
2 preD = list[list]
3
4 def topo_search(root):
5     """Topologische Suche im DAG"""
6     Q = collections.deque()
7     Q.push(root)
8
9     while Q:

```

¹⁰Knoten die man nicht entfernen darf, ohne den Graphen in mehrere Zusammenhangskomponenten zu teilen.

```

10     v = Q.pop()
11
12     processNode(v) # ...
13
14     for u in nexT[v]:
15         # Entferne die Kante aus dem Graphen
16         nexT[v].remove(u)
17         preD[u].remove(v)
18         if len(preD[u]) == 0:
19             Q.push(u)

```

Dieser Algorithmus lässt sich auch verwenden um zu Prüfen ob ein Zyklus in einem gerichteten Graphen ist, da sonst immer die Kantenmengen leer sein müssten.

Bei allgemeinen ungerichteten Graphen lässt sich dies leichter bestimmen indem wir überprüfen, ob in irgendeinem Moment ein Knoten einen bereits besuchten Knoten durch eine neue Kante erreichen kann.

6.2 Algorithmen auf Gewichteten Graphen

Bis jetzt haben wir nur ungewichtete Graphen betrachtet. Für gewichtete Graphen definieren wir $w \in R^E$ eine Abbildung, die jeder Kante eine Gewichtung zuordnet. Es gibt nun einige interessante Herausforderungen für gewichtete Graphen. Beispielsweise will man aus einem Graphen den Spannbaum mit minimalen Kantengewichten berechnen, oder kürzeste Wege zwischen zwei oder mehreren Punkten finden.

6.2.1 Kruskal's Algorithmus

Um einen minimalen Spannbaum zu finden, lässt sich Kruskal's Algorithmus verwenden. Der funktioniert so, dass man vorerst alle Kanten nach Gewicht sortiert, und dann immer minimale Kanten hinzufügt, wenn die zu verbindenden Knoten nicht schon in der gleichen Zusammenhangskomponente sind. Dafür lässt sich klug eine Union-Find Datenstruktur verwenden:

```

1 def kruskal(edges, n):
2     # Sortiere Kanten nach Gewichtung
3     edges.sort(key=lambda x: x[2])
4
5     E = []
6     UF = UnionFind(n) # Speichert Knoten 0..n-1
7
8     for a, b, w in edges:
9         i = UF.find(a)
10        j = UF.find(b)
11        if i != j:
12            E.append((a, b))
13            UF.union(i, j)

```

Recht ähnlich verläuft der Algorithmus von Prim, bei dem mit einem Knoten gestartet wird, und dann immer jeweils die kleinste Kante, die einen Knoten außerhalb des aktuellen Baumes hinzufügt, gewählt wird.

6.2.2 Kürzeste Pfade

Es gibt verschiedene Algorithmen um kürzeste Pfade im Graphen zu finden. Um in ungewichteten Graphen den kürzesten Pfad zwischen zwei Knoten zu finden, lässt sich Breitensuche verwenden, wie bereits erwähnt. Für gewichtete Graphen müssen aber andere Verfahren benutzt werden.

Wenn kürzeste Pfade von mehreren Knoten aus gesucht werden, kann ein weiterer Knoten hinzugefügt werden, der einen Nullkante zu den jeweiligen Knoten besitzt. Danach macht man einfach mit einem der folgenden Algorithmen weiter, die den kürzesten Pfad zwischen zwei Knoten berechnen.

Dijkstra Dijkstra's Algorithmus ist ein Greedy DP Algorithmus, der iterativ den Graphen traversiert, und dabei für jeden Knoten mittels seiner Kanten die Distanz zum Ursprungsknoten aktualisiert. Der Algorithmus funktioniert nur mit positive Kanten. Die Distanz zum Knoten wird als DP-Feld gespeichert. Kanten werden mittels einer Min-Priority Queue gewählt. Somit ergibt sich eine worst-case Laufzeit von $O(|E| + |V| \log |V|)$.

Bellman-Ford Der Algorithmus von Bellman und Ford lässt sich für negative Kanten verwenden, und kann benutzt werden um negative Zyklen in Graph zu finden. Bellman-Ford funktioniert, indem ein DP-Feld die Distanz zum Ursprungsknoten speichert. Die Distanz D_i wird zunächst für alle Knoten außer dem Ursprungsknoten auf ∞ gesetzt. Nun wird maximal $n - 1$ Mal jede Kante (i, j) mit Gewicht w_{ij} überprüft, ob $D_i + w_{ij} < D_j$ gilt. Wenn ja wird die jeweilige Distanz aktualisiert. Die Pfadlänge wird pro Iteration möglicherweise immer um eins erhöht, weswegen durch diesen Algorithmus auch ein $k \in [n - 1]$ langer Pfad gefunden werden kann. Wenn nach einer weiteren, n -ten, Iteration sich Distanzen aktualisieren, existiert ein negativer Zyklus. Es ergibt sich eine Laufzeit von $O(|V| \cdot |E|)$.

Wenn der kürzeste Weg im Graph zwischen allen möglichen Kombinationen von Knoten gesucht sind, lässt sich ein weiterer Algorithmus verwenden.

Floyd-Warschall Floyd's Algorithmus ist ein Algorithmus, der Bellman-Ford erweitert und dynamische Programmierung verwendet. Hier wird ein Feld $D_k(i, j)$ definiert, welches die kürzeste Weglänge von i nach j speichert. Nun werden n Iterationen durchgeführt, die jeweils alle Knotenpaare durchgehen, und das DP-Feld aktualisieren. Jede Iteration wird wieder die Pfadlänge erhöht, wenn es sich lohnt. Somit ergibt sich eine Laufzeit von $O(|V|^3)$.

6.3 Flussnetzwerke

Eine weitere Klasse an Problemen auf gerichteten Graphen sind Flussnetzwerke. Ein Netzwerk ist ein gerichteter Graph (V, E) mit Quelle¹¹ $s \in V$, sowie Senke¹² $t \in V$, und Kapazitäten $c_{ij} \in \mathbb{N}$ für $(i, j) \in E$. Ein Fluss im Netzwerk wird definiert durch Flusswert $f_{ij} \in \mathbb{Z}$ für $(i, j) \in E$. Dabei muss ein Fluss folgende Eigenschaften erfüllen:

1. Kapazitätskonformität $\forall (i, j) \in E : f_{ij} \leq c_{ij}$.
2. Antisymmetrie $\forall (i, j) \in E : f_{ij} = f_{ji}$, wenn definiert.
3. Flusserhaltung $\forall i \in V \setminus \{s, t\} : \sum_{j \in \text{pred}(i)} f_{ji} = \sum_{j \in \text{next}(i)} f_{ij}$.

Es gibt verschiedene Probleme auf Flussnetzwerken, beispielweise dem MAXIMUM-FLOW-Problem.

6.3.1 Maximum-Flow

Bei dem MAXIMUM-FLOW-Problem der Fluss gesucht wird mit dem maximalen Flussgesamtsumme $|f| := \sum_{j \in \text{pred}(t)} f_{jt} = \sum_{i \in \text{next}(s)} f_{sj}$. Um das Problem zu lösen lässt sich der Ford-Fulkerson- oder Edmonds-Karp-Algorithmus verwenden.

```
1 n : int
2 m : int
3 adj : list[list]
4 cap : list[int]
5 flow : list[int]
6
7 def ffek():
8     # Quelle und Senke
9     s, t = 1, n
10
11     while True:
12         # Berechne Pfad von s zu t im Residualgraphen
13         edges = bfs(s, t, n) # Tiefensuche bei Ford-Fulkerson
14         if not edges:
15             break
16
17         # Berechne kleinste Restkapazität auf dem Pfad
18         df = min(cap[i][j] - flow[i][j] for (i, j) in edges)
19         # Aktualisiere Fluss auf allen Kanten auf dem Pfad
20         for (i, j) in edges:
21             flow[i][j] += df
22             flow[j][i] -= df
23
```

¹¹Eng. source.

¹²Eng. sink.

```

24     # Berechnung des max. Flusswerts nach Konvergenz
25     fmax = sum(flow[i][t] for i in range(n))
26
27 def bfs(s, t, n):
28     done = False # Stoppe wenn t gefunden
29     b = [False] * n
30     parent = [-1] * n
31     visited[s - 1] = True
32     queue = collections.deque([s])
33     while queue and not done: # ...
34         i = queue.popleft()
35         for j in adj[i-1]:
36             if not b[i-1] and cap[i-1][j-1] > flow[i-1][j-1]: # Residualgraphskriterium
37                 b[i-1] = True
38                 parent[j-1] = i
39                 if j == t:
40                     done = True
41                     break
42                 queue.append(j)
43     return getPath(t, parent)
44
45 # Findet Pfad von n zu t mittels Rückwärtstraversierung über parent.
46 def getPath(t, parent):
47     edges = []
48     i = t
49     while parent[i-1] != -1:
50         edges.append((parent[i-1], i))
51         i = parent[i-1]
52     return edges

```

Sind mehrere Quellen vorhanden, muss man einfach nur eine weitere Quelle hinzufügen, die als eigentliche Quelle mit unendlicher Kapazität zu den originalen Quellen verbunden ist.

Limitiert ein Knoten selbst das Netzwerk, da er eine Kapazität besitzt, muss man einfach einen weiteren Knoten hinzufügen, der vor dem Knoten gelegen wird, und jene Kapazität als Verbindung zum Ursprungsknoten besitzt.

Existieren Kanten mit entgegengesetzter Richtung, die die Antisymmetrie verletzen, kann man einfach einen Knoten in die Gegenkante einfügen.

Besitzt das Netzwerk Mehrfachkanten, kann man die Kanten zusammenfassen indem man ihre Kapazitäten addiert.

Der Algorithmus von Edmonds und Karp besitzt Laufzeit $O(n \cdot m^2)$, das MAXIMUM-FLOW-Problem lässt sich jedoch sogar in $O(n \cdot m)$ oder $O(m^{1+o(1)})$ lösen.

6.3.2 Minimum-Cut

Ein weiteres Problem ist die Suche nach einer Partition $\{S, T\}$ von V mit $s \in S$ und $t \in T$, sowie minimaler $c(S, T) := \sum \{c_{ij} \mid (i, j) \in E \wedge i \in S \wedge j \in T\}$.

Um das Problem zu lösen, lässt sich der maximale Fluss finden, da $|f| = c(S, T)$. Dann kann man jeweils im berechneten Residualgraphen G_f nach Knoten suchen, die von s bzw. t (mit umgedrehten Kanten) erreichbar sind.

6.3.3 Maximum-Bipartite-Matching

Flussnetzwerke lassen sich auch verwenden, um das größtmögliche Matching zwischen Knoten in einem bipartiten Graphen zu finden. Dabei ist ein Matching eine Teilmenge der Kanten, sodass Knoten nur mittels genau einer Kante verbunden werden können.

Das Problem lässt sich lösen, indem man ein Netzwerk konstruiert, welches eine Quelle hat, die zu allen Knoten aus der ersten Klasse verbunden ist, welche zu den jeweiligen Knoten aus der anderen Klasse verbunden sind, die zur Senke verbunden sind. Alle Kanten haben Kapazität 1. Berechnet man nun den maximalen Fluss werden genau die Kanten gefüllt, die in einer maximalen Matching verwendet werden.

7 Range Queries

Range Queries beschäftigen sich mit der Frage wie man für $a \in \mathbb{N}^n$ und $i, j \in [n], i \leq j$, sowie $f : \mathbb{N}^d \rightarrow \mathbb{R}$ mit $d \in [n]$ alle möglichen $f(a[i..j])$ berechnet. Dabei ist der naive Ansatz $O(n)$ langsam, jedoch gibt es Ansätze, um eine Range Query schneller zu bearbeiten.

7.1 Statische Daten

Wenn sich das Feld a nicht verändert, eignen sich folgende Ansätze.

Ergebnistabelle Berechne alle n^2 Einträge $f(a[i..j])$ vor und speichere sie in einem 2D Array. Vorbereitung dauert mindestens $\Omega(n^2)$, sowie müssen $\Theta(n^2)$ Werte vorberechnet werden. Dafür ist die Auswertung immer $O(1)$.

Präfixsummenarray Funktioniert nur für $f = \sum$. Speichere ein weiteres Feld $\sigma \in \mathbb{N}^n$ mit $\sigma_i = \sum_{j \in [i]} a_j$ für $i \in [n]$. Nun um $f(a[i..j])$ zu berechnen, gib $\sigma_j - \sigma_{i-1}$ zurück, wenn $i > 1$, sonst σ_j . Die nötige Laufzeit und Speicher für die Vorbereitung, jedoch kann eine Query mit $O(1)$ bearbeitet werden.

Sparse Table Für idempotente Funktionen wie $\min$ oder $\max$ eignen sich sog. *Sparse Tables*, bei denen nur für Teilsequenzen mit einer Zweierpotenz als Länge, Werte vorberechnet werden. Somit füllt sich eine Tabelle $M \in \mathbb{N}^{n \times n}$ oder ein Binärbaum mit $n \log n$ Werten mit $M[i][i] = a_i$, und $M[i][i+2^p-1] = f(M[i][i+2^{p-1}-1], M[i+2^{p-1}][i+2^p-1])$. Bei idempotenten Funktionen ist die Auswertung möglich in $O(1)$, sonst in $O(\log n)$.

7.2 Dynamische Daten

Daten können einzeln, *Point Update*, sowie als Teilsequenz, *Range Update*, verändert werden. Eine Datenstruktur für Range Queries sollten in diesen Fällen nicht von vorne berechnet werden, weswegen sich datenstrukturen anbieten, die Datenveränderungen ermöglichen.

Naiver Ansatz Wenn man keine Hilfsdatenstruktur verwendet, kommt man auf $O(n)$ pro Query, jedoch auf $O(1)$ pro Update.

Präfixsummenarray Beim Präfixsummenarray muss man in einem Update leider $O(n)$ Werte verändern, die Query ist aber konstant schnell.

Stattdessen lassen sich Baumstrukturen verwenden, mit denen Query und Update in $O(n \log n)$ ermöglicht werden.

7.2.1 Fenwick-Bäume

Fenwick-Bäume ermöglichen schnellere Sum Range-Queries auf dynamischen Daten. In einem Fenwick-Baum speichern Knoten die Gesamtsumme einer gewissen Teilsequenz von a . Will man nun eine Range-Query $f(a[i..j])$ berechnen, wählt man Additive und Subtraktive aus dem Baum und erhält das Ergebnis in $O(\log n)$. Das Schöne daran ist, dass man ein Update so implementieren kann, dass das Update vom Kindknoten aus nach Oben backpropagiert wird, und somit auch $O(\log n)$ erreicht. Der Baum kann in $O(n)$ erstellt werden, mehr dazu im Competitive Programmer's Handbook: <https://cses.fi/book/book.pdf#section.9.2>.

Der Nachteil des Fenwick-Baumes, ist es, dass wie beim Präfixsummenarray, keine anderen Range-Queries unterstützt werden, und, dass Range-Updates nur punktwise funktionieren, man käme also auf $O(n \log n)$.

7.2.2 Segmentbäume

Um die Probleme von Fenwick-Bäumen zu lösen, kann man Segmentbäume verwenden, die das generelle Prinzip des Fenwick-Baums generalisieren. Statt genau n Summen zu speichern, die $[n]$ partitionieren, verwendet man $2n - 1$ Einträge, die jeweils alle möglichen $f(a[i, i])$, sowie $f(a[i, i + 1])$, $f(a[i, i + 3])$, $f(a[i, i + 7])$, bis $f(a[i, i + n - 1])$ repräsentieren. Die inneren Knoten speichern also $f(l, r)$ wobei l, r die jeweiligen Kinder des Knoten sind. Um nun $f(a[i, j])$ zu berechnen, kann der folgende Algorithmus verwendet werden:

1. Starte mit Intervall $[l, r] \leftarrow [1, n]$, und $F =$ dem neutralen Element von f .
2. Überlappt $[l, r]$ vollständig mit $[i, j]$? Dann $F \leftarrow f(F, f(a[l, r]))$.
3. Überlappt $[l, r]$ überhaupt nicht mit $[i, j]$? Dann lasse F gleich.
4. Überlappt $[l, r]$ teilweise mit $[i, j]$? Die Mitte ist $\lfloor (l + r)/2 \rfloor$. Mache mit $[l, m]$ und $[m + 1, r]$ rekursiv weiter.

Insgesamt müssen also nur $O(\log n)$ Einträge ausgewertet werden. Das Point-Update funktioniert genau wie bei den Fenwick-Bäumen – ein Update wird den jeweiligen Elternknoten weitergegeben.

Das Range Update würde naiv im worst case $2n - 1$ Updates nötig machen, weswegen sogenannte *Lazy Updates* meistens verwendet werden, die Updates erst ausführen, wenn sie bei einer Range-Query relevant wären. Man kann sowas implementieren, indem man die Updates in den Blattknoten vormerkt, und dann bei der Auswertung einer Query, alle relevanten Updates backpropagiert. Somit bleibt man bei $O(\log n)$ bei einer Range Query, und ein Range Update kann mit minimalen Overhead $O(\log n)$ realisiert werden.